

Linux Operating Systems — Assignment

Student: Nithin Shet N
Reg. No.: 261100690025

1. What If There Were No Operating System?

Strip away the operating system and a computer isn't really a "computer" in any useful sense — it's just silicon that can execute instructions if you feed it the right voltages in the right order. Every single thing we take for granted, from clicking an icon to saving a file, exists because the OS sits between us and the raw hardware, translating our intentions into something the machine can act on. Without it, the burden of managing hardware falls entirely on whoever wrote the program, which is exactly why early computers ran one program at a time, written by people who understood the machine down to the register level.

Problems a user would run into without an OS :

- Multitasking — only one program can occupy the processor at any moment, so switching tasks means stopping one program entirely before starting another.
- File system — data has nowhere structured to live; it's just raw bytes on a disk with no folders, names, or way to find anything later.
- Protection — any running program can read or overwrite any part of memory, including areas belonging to something else entirely.
- Standard way to talk to hardware — every application would need its own driver code for the keyboard, disk, and display.
- User interface — there's no shell, no desktop, no prompt; interacting with the machine means writing and loading raw machine code.

Three OS features, demonstrated on Linux

Problem Without an OS	How Linux Solves It	Command to See It
Can't run more than one program	The kernel's scheduler slices CPU time between processes so they appear to run simultaneously	ps aux — lists every active process with its PID and state
No organized storage	Linux provides a hierarchical file system with directories, permissions, and metadata	mkdir project && ls -l — creates and lists a folder
No memory or data protection	Each process gets its own virtual address space; files carry permission bits	free -h shows memory use; chmod 600 file restricts access

Without OS vs. With OS

Aspect	Without an OS	With an OS (Linux)
Running programs	One program at a time, run to completion	Dozens of processes scheduled concurrently
Storage	Raw disk blocks, no organization	Structured file system (ext4, etc.) with folders and metadata
Memory	Shared address space; anything can overwrite anything	Isolated virtual memory per process
Security	None — no concept of permissions or users	File permissions, user accounts, access control
Usability	Requires writing raw machine-level code	Simple commands or a graphical interface

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide.”

Cybersecurity lab isn't a one-OS operation. The tools a penetration tester needs are almost the opposite of what a forensics analyst needs. So the practical approach is to assign a purpose-built distribution to each role rather than trying to force one system to do everything.

Distribution	Best Role	Why It Fits
Kali Linux	Penetration Testing	Ships with hundreds of pre-configured offensive tools (Nmap, Metasploit, Burp Suite); the de facto standard for red-team work
Parrot Security OS	Testing + Privacy/Dev	Lighter than Kali, Debian-based, bundles the same class of testing tools plus sandboxing and privacy tools — good for a lab that also does secure development
CAINE	Digital Forensics	Built around a strict rule of not writing to the disk being examined, which is critical for keeping evidence legally admissible
Tails	Privacy / Anonymous Ops	A live, amnesiac OS — routes everything through Tor and wipes itself on shutdown, leaving no trace on the host machine
Security Onion	Security Monitoring	A ready-made network monitoring stack (Zeek, Suricata, Elastic) for watching traffic and triaging alerts across the lab.

Recommendations by role.

Red team / offensive testing: Kali Linux for its breadth of tools and community support; Parrot as a lighter alternative if hardware is limited.

- Blue team / forensics: CAINE for evidence-safe disk analysis, ideally paired with write-blockers at the hardware level.
- Privacy-sensitive or anonymous research: Tails, run live off a USB so nothing persists on the lab machine.
- Network monitoring / SOC-style operations: Security Onion as the central dashboard for intrusion detection and log correlation.

3.Exit status:

Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using ``$?``.

Every time a Linux command finishes, it hands the shell a small number called its exit status, and that number is the only thing a script has to go on when deciding whether the previous step actually worked. By convention, 0 means success and anything from 1–255 means something went wrong, though the exact non-zero value can hint at what kind of failure it was.

Command	Exit Status	Meaning
ls /etc	0	Success — directory exists and was listed
ls /no_such_folder	2	Failure — the directory doesn't exist
mkdir new_folder	0	Success — directory created
mkdir /root/blocked	1	Failure — permission denied
grep root /etc/passwd	0	Success — the pattern was found
grep zzzznotfound /etc/passwd	1	Failure — command ran fine, but nothing

A script has no eyes — it can't glance at the terminal and notice that a folder wasn't found. The exit status is its only feedback signal, and that single number lets it make decisions the way a person would after reading an error message:

- Branch its logic — an if command; then ... else ... structure runs different code depending on success or failure.
- Chain commands conditionally — `cmd1 && cmd2` runs the second command only if the first succeeded;
`cmd1 || cmd2` runs the second only if the first failed, which is handy for fallback logic or logging errors.
- Fail safely — a script that ignores exit status might carry on after a failed step (like a backup that silently didn't run) and cause bigger problems downstream; checking `$?` lets it stop or retry instead.

In short, exit status is what turns a sequence of commands into something that can actually be called automation without it, a script would be flying blind after every single line.